Brickts SQL course — Week 3 — Make time explicit

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 3 — Make time explicit

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 15: Day 15 — Query UTC instants with half-open windows
- Day 16: Day 16 — Use CTEs to make query stages readable

- Day 17: Day 17 — Add window calculations without collapsing rows
- Day 18: Day 18 — Compare neighboring observations with LAG and LEAD
- Day 19: Day 19 — Bound valid coverage instead of bridging gaps
- Day 20: Day 20 — Build explicit conditional summaries and pivots
- Day 21 (weekly challenge): Day 21 — Week 3 challenge — a time-series quality report

---

Day 15 — Query UTC instants with half-open windows

Previous: Day 14 · **Week 3 · 45–75 minutes.** Prerequisites: days 1–14, or equivalent skills.

Goal

Select exact time windows, distinguish an instant from its display timezone, and explain why adjacent windows should share a boundary without sharing observations.

Before you start

Connect to the course database. Run `SHOW timezone;` and record the result. The fixture begins at `2026-01-05T00:00:00Z` and stops before `2026-01-07T00:00:00Z`. Each complete point has 576 observations at five-minute cadence.

Concept

A `timestamptz` stores an instant. PostgreSQL displays it using the session timezone; changing that setting does not move the stored observation. A timestamp literal with an explicit `+00` offset makes the intended UTC instant clear. A local timestamp without an offset needs a timezone interpretation, and daylight-saving changes can make local clock labels ambiguous or nonexistent.

Use a half-open window: `ts >= start AND ts < end`. An observation exactly at the end belongs to the next window. This lets adjacent reports partition time cleanly. `BETWEEN` includes both endpoints, which can double count a boundary when used for adjacent windows.

Tiny example

```
SET TIME ZONE 'UTC';
SELECT count(*) AS stored_pressure_rows
FROM course.history
WHERE role = 'duct_static'
  AND ts >= TIMESTAMPTZ '2026-01-05
         08:00:00+00'
  AND ts <  TIMESTAMPTZ '2026-01-05
         09:00:00+00';
```

Expect 34 rows: three pressure points would normally contribute 36, but two observations are absent. This is a count of stored rows, not a duration calculation.

SQL challenge

Build two adjacent one-hour reports for pressure observations, grouped by site and point identity. Include the start and end instants in your report notes. Identify which first-window point has 10 rows rather than the usual 12, then retrieve its equipment metadata through identities.

Verify that the sum of both adjacent-window counts equals the count for their combined two-hour window. State whether the boundary observation appears in the earlier or later report. Keep quality counts separate from missing-row counts.

Experiment

Display one known timestamp with `SET TIME ZONE 'UTC'`, then with `SET TIME ZONE 'America/Chicago'`. The UTC midnight fixture instant displays as the preceding evening in Chicago. Confirm its identity and numeric value stay the same, then restore UTC. Compare a `BETWEEN` one-hour count with the half-open count: the inclusive endpoint adds one observation per pressure point at 09:00.

Acceptance checks

- The 08:00–09:00 UTC pressure count is 34, and your affected point has 10 rows.
- Adjacent half-open windows reconcile with their combined window.
- Your notes distinguish UTC instants, local display, and inclusive/exclusive boundaries.

Stretch and reflection

Convert an instant with `AT TIME ZONE` and inspect the resulting type using `pg_typeof`. Why does a local calendar day require timezone-aware boundary construction? Read week 3 references before assuming every local day is 24 hours.

Previous: Day 14 ·

Day 16 — Use CTEs to make query stages readable

Previous: Day 15 · **Week 3 · 45–75 minutes.** Prerequisites: days 1–15, or equivalent skills.

Goal

Break a report into named relational stages with a common table expression, or CTE. Explain each stage's grain and verify that an early filter matches the report's question.

Before you start

Review `WHERE`, grouping, and the good/non-null calculation policy. A CTE is introduced with `WITH`. It is available to the statement that follows; it does not create a durable table or a new stored copy of the fixture.

Concept

A complicated query is easier to review when selection, quality filtering, and summary have clear names. Each CTE is a relation with named columns. A later stage can consume an earlier one. Names should describe the population or result, such as `usable_power`, rather than merely `step1`.

CTEs are a readability tool, not a guarantee that PostgreSQL executes each stage separately or that a query becomes faster. The optimizer can change the execution strategy. First ensure the statement answers the right question; execution plans come next week.

Tiny example

```
WITH usable_power AS (
    SELECT site_id, point_id, ts, value
    FROM course.history
    WHERE role = 'kw'
      AND quality = 'good'
      AND value IS NOT NULL
)
SELECT site_id, count(*) AS usable_rows,
         avg(value) AS mean_kw
FROM usable_power
GROUP BY site_id
ORDER BY site_id;
```

The named stage has one row per usable power observation. The result has one row per site, with 1,151 usable rows at north and 576 at south. The average is a sample-weighted mean power across observations at that site; it is neither energy nor an equipment total.

SQL challenge

Write a CTE report for temperature observations during Jan 5 UTC. Make the time selection and quality policy visible, then summarize by site, equipment identity, and role. Choose two temperature roles and show usable count and mean in degrees Celsius. State each stage's grain before executing.

Write a second CTE that counts stored observations per pressure point for that same day, then returns only short groups against the 288-row expectation. The affected pressure point should have 286 rows. Resolve its equipment through metadata after identifying its point identity.

Experiment

Move the quality filter after a first summary in a draft. Can a grouped count still tell you which individual values were unusable if you discarded their quality field? Record the

missing information. Compare with a design that preserves stored count and usable count as separate aggregates.

Inspect the CTE alone by temporarily selecting its named columns from it. Limit and order the inspection so your evidence is small and repeatable.

Acceptance checks

- Your stages have explicit grains and do not mix incompatible units.
- The power demonstration returns usable counts 1,151 and 576.
- The Jan 5 pressure report finds one 286-row point and does not infer its identity from a label.

Stretch and reflection

Create two related CTEs in one statement and explain their dependency. Which name makes a mistaken filter easier to notice? Read week 3 references for `WITH` scope and optimizer behavior.

Previous: Day 15 ·

Day 17 — Add window calculations without collapsing rows

Previous: Day 16 · **Week 3 · 45–75 minutes.** Prerequisites: days 1–16, or equivalent skills.

Goal

Use `OVER`, `PARTITION BY`, and ordering to add context to individual observations. Distinguish a window calculation from a grouped summary.

Before you start

Review the grain of `course.history`. A pressure history contains one observation per point and instant. Partitioning by site and point keeps independent sensors separate even when timestamps and roles match.

Concept

`GROUP BY` collapses rows into one output per group. A window function calculates across a related set of rows while retaining each input row. `PARTITION BY` chooses independent populations. `ORDER BY` inside `OVER` defines the order used by functions such as `row_number`; it is separate from the final result's display order.

A window aggregate without an internal ordering, such as `count(*) OVER (PARTITION BY ...)`, can provide a full-partition count on every observation. Adding an ordering changes the default frame for many aggregates, often producing a running calculation. State the frame explicitly when learning running calculations, and do not assume every `OVER` clause means the same population.

Tiny example

```
SELECT site_id, point_id, ts,
       row_number() OVER (
           PARTITION BY site_id, point_id
         ORDER BY ts
       ) AS observation_number,
       count(*) OVER (
           PARTITION BY site_id, point_id
       ) AS point_stored_rows
FROM course.history
WHERE role = 'duct_static'
ORDER BY site_id, point_id, ts
LIMIT 8;
```

The excerpt still has observation grain. Across the full result, one point's stored-row count is 574 and the other two are 576. The displayed point depends on UUID ordering; inspect all three deliberately before drawing a conclusion.

SQL challenge

Use a CTE to number power observations within each site and point. Return the first and last observation from every point with its original timestamp, value, and quality. Do not choose a point by the smallest UUID or combine multiple points into one partition.

Produce a separate grouped count for each power point and reconcile it with the window partition counts. All three have 576 stored rows even though one has an unusable value. Explain why row numbering does not repair that measurement.

Experiment

Remove `point_id` from the partition and inspect a bounded excerpt. Several power points now share a site population, and equal timestamps create ties. Identify why row numbers no longer mean position within one sensor history. Restore the intended partition and order.

Compare a full-partition count with an explicitly running `count(*) OVER (PARTITION BY site_id, point_id ORDER BY ts ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`. Predict each value at the third observation.

Acceptance checks

- First/last reports retain identities and return six power rows in total.
- Counts and row numbers have different meanings you can explain.
- Your partitions preserve individual point histories and your final output order is

explicit.

Stretch and reflection

Why must a deterministic tie breaker be added if ordering columns are not unique? Read week 3 references for window frames and function behavior.

Previous: Day 16 ·

Day 18 — Compare neighboring observations with LAG and LEAD

Previous: Day 17 · **Week 3 · 45–75 minutes.** Prerequisites: days 1–17, or equivalent skills.

Goal

Use the previous or next observation to detect unusual spacing while preserving the identity and time ordering of each sensor history.

Before you start

Set your session timezone to UTC. Review window partitions. The fixture uses five-minute cadence, but two pressure observations are absent on Jan 5. A stored row with a null value remains a row; it does not create a timestamp gap by itself.

Concept

`lag(ts)` returns the previous row's timestamp within the ordered partition; `lead(ts)` returns the next. The first or last row has no neighbor in that direction, so the function returns null by default. That boundary null is different from a missing measurement.

Timestamp subtraction produces an interval. A gap detector compares that interval with an expected cadence. It detects an interruption between observed rows but does not establish the values during the interruption. The neighbor population matters: filtering to good rows before `lag` measures gaps between usable observations; retaining every stored row measures storage cadence.

Tiny example

```
WITH neighbors AS (
    SELECT site_id, point_id, ts,
           lag(ts) OVER (
               PARTITION BY site_id,
          point_id ORDER BY ts
           ) AS previous_ts
    FROM course.history
    WHERE role = 'duct_static'
)
SELECT site_id, point_id, previous_ts, ts,
       ts - previous_ts AS observed_gap
FROM neighbors
WHERE ts - previous_ts > INTERVAL '5
         minutes'
ORDER BY site_id, point_id, ts;
```

Exactly one row has a gap of fifteen minutes: the affected pressure point jumps from 08:20 to 08:35 UTC on Jan 5. This implies two expected interior instants are absent, not fifteen minutes of known pressure behavior.

SQL challenge

Create a bounded neighbor report for `kw` observations. Include current quality and value along with previous and next timestamps. Verify that the stored power rows retain normal cadence around the bad/null observation at 15:10 UTC on Jan 5. Then compare with a version whose window input

contains only usable power observations. Explain the different question each report answers.

Use point identities in partitions and retrieve equipment metadata only through the established relationship. Describe the expected behavior at the first and last fixture observations.

Experiment

Restrict the pressure CTE's input to an 08:30–09:00 window and rerun the detector. Its first visible row no longer sees 08:20. Compare that with calculating neighbors first and applying the report window afterward. Explain why a query can miss a boundary gap without any SQL error.

Acceptance checks

- The pressure detector reports one 15-minute observed gap at the expected instants.
- Your power report distinguishes storage cadence from usable-value cadence.
- Your notes account for window boundaries and neighbor nulls.

Stretch and reflection

What additional expected timestamp series would reveal missing observations before the first stored row or after the last? Outline the needed information without assuming `lag` can create absent rows. Read week 3 references for offset window functions.

Previous: Day 17 ·

Day 19 — Bound valid coverage instead of bridging gaps

Previous: Day 18 · **Week 3 · 45–75 minutes.** Prerequisites: days 1–18, or equivalent skills.

Goal

Translate a declared sample-support convention into bounded coverage, and report unknown time separately. Counts become durations only after you choose a support rule.

Before you start

Read the fixture's convention in the contract: a usable sample can represent at most 300 seconds, clipped to the requested half-open window. A long gap cannot extend the preceding value indefinitely. Today studies coverage, not proof of a process state between measurements.

Concept

Treat a usable observation at `ts` as supporting a forward interval that ends at the earliest of the next observation, `ts + 5 minutes`, and the report end. Clip the start to the report start. A positive interval contributes coverage; a bad/null row contributes none. A report beginning between samples may need the immediately preceding observation to assess the opening interval.

This is a teaching convention. It does not demonstrate that the physical value stayed constant. Scheduled fan hours, hours inferred from observed fan status, pressure coverage, and unknown intervals are distinct quantities. Never substitute one for another without declaring why it answers the question.

Tiny example

```
WITH pressure_neighbors AS (
    SELECT site_id, point_id, ts, value,
         quality,
           lead(ts) OVER (
               PARTITION BY site_id,
         point_id ORDER BY ts
           ) AS next_ts
    FROM course.history
    WHERE role = 'duct_static'
)
SELECT site_id, point_id, ts, next_ts,
       least(ts + INTERVAL '5 minutes',
            coalesce(next_ts, ts +
         INTERVAL '5 minutes'),
             TIMESTAMPTZ '2026-01-05
         09:00:00+00') - ts
           AS supported_interval
FROM pressure_neighbors
WHERE site_id = 'SYN-NORTH'
  AND ts = TIMESTAMPTZ '2026-01-05
         08:20:00+00'
  AND quality = 'good' AND value IS NOT
         NULL
ORDER BY point_id;
```

Both northern pressure rows support five minutes. One next observation is fifteen minutes away; its support still ends at 08:25.

SQL challenge

Report usable pressure coverage per point for the exact 08:00–09:00 UTC window. State the grain, quality rule, support convention,

and seconds unit before writing the formula. Report requested seconds, covered seconds, and unknown seconds together. Ensure support cannot extend outside the window or become negative.

The affected point should cover 3,000 of 3,600 seconds, leaving 600 unknown. Each other pressure point covers all 3,600 seconds. Explain why the fifteen-minute neighbor gap does not mean fifteen minutes are all unknown under this convention.

Experiment

Change the report start to 08:22. Include the preceding 08:20 observation when constructing support, then clip it to the new start. Compare with filtering all input rows to `ts >= start`. The latter loses legitimate opening support. Record this boundary limitation explicitly.

Acceptance checks

- Your one-hour report produces the stated 3,000/600 and 3,600/0 coverage pairs.
- No usable row represents more than 300 seconds.
- Bad/null values and unsupported gaps never become known coverage.

Stretch and reflection

How would irregular sampling change the support policy? Describe a requirement and a test before choosing interpolation. Read week 3 references for date arithmetic and window functions.

Previous: Day 18 ·

Day 20 — Build explicit conditional summaries and pivots

Previous: Day 19 · **Week 3 · 45–75 minutes.** Prerequisites: days 1–19, or equivalent skills.

Goal

Use aggregate `FILTER` clauses to summarize quality and place selected roles beside each other, while proving the input grain supports the pivot.

Before you start

Review grouped aggregates and role/unit metadata. A long table has one row per point and instant; a pivot can have one row per equipment and instant with separate columns for different roles. Each numeric output still needs its own unit.

Concept

`count(*) FILTER (WHERE ...)` counts only the rows matching that aggregate's condition. Other aggregates in the same group can use different filters. This preserves a visible distinction between stored rows, usable rows, and bad/null rows.

A conditional pivot often uses `max(value) FILTER (WHERE role = ...)` to extract a role's value. Here `max` is safe only after establishing at most one selected point per equipment and role at each instant. It must not silently choose the highest of duplicate same-role sensors. Missing source rows and excluded bad values both produce null pivot cells, so retain a quality report to explain them.

Tiny example

```
SELECT site_id, equipment_id, ts,
       max(value) FILTER (
           WHERE role = 'duct_static'
             AND quality = 'good' AND
         value IS NOT NULL
       ) AS duct_static_pa,
       max(value) FILTER (
           WHERE role = 'duct_static_sp'
             AND quality = 'good' AND
         value IS NOT NULL
       ) AS duct_static_sp_pa
FROM course.history
WHERE site_id = 'SYN-NORTH'
  AND ts = TIMESTAMPTZ '2026-01-05
       08:25:00+00'
GROUP BY site_id, equipment_id, ts
ORDER BY equipment_id;
```

The result has two equipment rows. One pressure cell is null because that observation is absent; its setpoint is still 250 Pa. The other AHU has its own values.

SQL challenge

Build a daily UTC quality report per site and role, with stored count, usable count, and bad/null count using explicit filters. Identify the single unusable `kw` sample without turning it into a zero.

Build a bounded instantaneous pivot for `fan_status`, `fan_speed_pct`, and `kw` with descriptive aliases. Show each role's unit in your report notes. Compare with the same rows from `course.ahu_5m`, which provides all fourteen teaching roles. This view does not resample time or fill absent values.

Experiment

Inspect the Jan 5 15:10 UTC power pivot. One equipment's power cell is null even though its fan-status observation is present. Explain why an aggregate across available values cannot establish the missing power value. Then count `course.ahu_5m`: it has 1,728 equipment/instant rows despite the two missing pressure samples.

Acceptance checks

- Your pivot retains equipment identity and has one row per equipment/instant.
- Your quality report accounts for the one bad/null power observation.
- You establish the starter fixture's single-point-per-role assumption before using `max` as extraction.

Stretch and reflection

Compare a conditional aggregate with `CASE` inside an aggregate. Which makes your quality rule easier to review? Read week 3 references for aggregate filters and conditional expressions.

Previous: Day 19 ·

Day 21 — Week 3 challenge — a time-series quality report

Previous: Day 20 · **Week 3 review · 2–4 hours.** Prerequisites: days 15–20.

Project brief

Produce a time-series quality report for the two-day synthetic fixture. Use precise UTC half-open windows and separate observations, measurement usability, and supported duration. Work independently with your earlier lessons and references; allow 2–4 hours.

The report should help a colleague decide where a later engineering calculation would have enough evidence. It should state its limits clearly: a normal sample count is not proof of valid values, and known schedule hours are not observed run hours. Keep raw evidence bounded and preserve every fixture row.

Acceptance criteria

- Report per-point stored and usable counts with site/equipment identity and role/unit context.
- Identify the affected pressure point and its Jan 5 08:20→08:35 UTC spacing, including the two absent expected instants.
- Distinguish the one bad/null power sample from an absent row.
- Quantify pressure coverage for Jan 5 08:00–09:00 UTC under the declared maximum-300-second support convention; the affected point has 3,000 covered and 600 unknown seconds.
- Adjacent half-open report windows reconcile without sharing boundary observations.
- Any pivot has a documented single-point-per-role assumption and leaves unknown values visible.

Deliverables

Submit saved SQL, compact actual results, and a report explaining at least one count issue, one quality issue, and one boundary decision. Include grain and units for every result. Mark output observed or not run; record an unmet check explicitly.

Review questions

What does the first visible row know about the prior interval? Why can a bad-value row preserve cadence? Which time is known, supported by a convention, scheduled, or unknown? How would duplicate sensors change the report?

Attempt this independently before consulting week 3 references. No worked implementation or calculation sequence is supplied.

Previous: Day 20 ·

Primary reading — Week 3

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 3

- Date/time types
- Date/time functions
- Window functions tutorial
- Window function details

- WITH